

Mocking Dependencies in Spring Tests using Mockito

Exercise 1: Mocking a Service Dependency in a Controller Test

****Task:**** Write a unit test for a Spring controller that uses a service to fetch data. Mock the service dependency using Mockito.

Step-by-Step Solution:

1. ****Create the User Entity:****

```
@Entity
public class User {
    @Id
    private Long id;
    private String name;
    // getters and setters
}
```

2. ****Create the UserService:****

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
    public User getUserById(Long id) {
        return userRepository.findById(id).orElse(null);
    }
}
```

3. ****Create the UserController:****

```
@RestController @RequestMapping("/users")
public class UserController {
    @Autowired
    private UserService userService;
    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }
}
```

4. ****Create the UserControllerTest:****

```
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
```

```

import org.springframework.test.web.servlet.MockMvc;
@WebMvcTest(UserController.class)
public class UserControllerTest {
    @Autowired
    private MockMvc mockMvc;
    @MockBean
    private UserService userService;
    @Test
    void testGetUser() throws Exception {
        User user = new User();
        user.setId(1L);
        user.setName("John");
        when(userService.getUserById(1L)).thenReturn(user);
        mockMvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.name").value("John"));
    }
}

```

Exercise 2: Mocking a Repository in a Service Test

****Task:**** Write a unit test for a Spring service that uses a repository to fetch data. Mock the repository dependency using Mockito.

Step-by-Step Solution:

1. ****Create the User Entity:****

```

@Entity
public class User {
    @Id
    private Long id;
    private String name;
    // getters and setters
}

```

2. ****Create the UserRepository:****

```

public interface UserRepository extends JpaRepository<User, Long> {
}

```

3. ****Create the UserService:****

```

@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
    public User getUserById(Long id) {
        return userRepository.findById(id).orElse(null);
    }
}

```

4. ****Create the UserServiceTest:****

```

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;
import java.util.Optional;
import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
public class UserServiceTest {
    @Mock
    private UserRepository userRepository;
    @InjectMocks
    private UserService userService;
    public UserServiceTest() {
        MockitoAnnotations.openMocks(this);
    }
    @Test
    void testGetUserById() {
        User user = new User();
        user.setId(1L);
        user.setName("John");
        when(userRepository.findById(1L))
            .thenReturn(Optional.of(user));
        User result = userService.getUserById(1L);
        assertEquals("John", result.getName());
    }
}

```

Exercise 3: Mocking a Service Dependency in an Integration Test

****Task:**** Write an integration test for a Spring Boot application that mocks a service dependency using Mockito.

Step-by-Step Solution:

1. ****Create the User Entity:****

```

@Entity
public class User {
    @Id
    private Long id;
    private String name;
    // getters and setters
}

```

2. ****Create the UserService:****

```

@Service
public class UserService { @Autowired
    private UserRepository userRepository;
    public User getUserById(Long id) {
        return userRepository.findById(id).orElse(null);
    }
}

```

```
}  
}
```

3. ****Create the UserController:****

```
@RestController  
@RequestMapping("/users")  
public class UserController {  
    @Autowired  
    private UserService userService;  
    @GetMapping("/{id}")  
    public ResponseEntity<User> getUser(@PathVariable Long id) {  
        return ResponseEntity.ok(userService.getUserById(id));  
    }  
}
```

4. ****Create the UserIntegrationTest:****

Write code for this.

```
import static org.mockito.Mockito.when;  
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;  
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;  
import org.junit.jupiter.api.Test;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;  
import org.springframework.boot.test.context.SpringBootTest;  
import org.springframework.boot.test.mock.mockito.MockBean;  
import org.springframework.test.web.servlet.MockMvc;  
@SpringBootTest  
@AutoConfigureMockMvc  
public class UserIntegrationTest {  
    @Autowired  
    private MockMvc mockMvc;  
    @MockBean  
    private UserService userService;  
    @Test  
    void testGetUser() throws Exception {  
        User user = new User();  
        user.setId(1L);  
        user.setName("John");  
        when(userService.getUserById(1L)).thenReturn(user);  
  
        mockMvc.perform(get("/users/1"))  
            .andExpect(status().isOk())  
            .andExpect(jsonPath("$.name").value("John"));  
    }  
}
```